

并行与分布式计算

-- 命名

Naming命名

本质

名称用于表示分布式系统中的实体。要对实体进行操作，我们需要在访问点访问它。访问点是通过地址命名的实体。

注意

实体 E 的位置独立名称与 E 提供的访问点的地址无关。

Identifiers标识符

Pure name 纯名称

完全没有意义的名称；它只是一个随机字符串。纯名称只能用于比较。

标识符：具有某些特定属性的名称

1. 标识符最多引用一个实体。
2. 每个实体最多由一个标识符引用。
3. 标识符始终引用同一个实体（即，它永远不会被重复使用）。

观察

标识符不一定是纯名称，即它可能有内容。

Broadcasting广播

广播 ID，请求实体返回其当前地址

- ▶ 永远无法扩展到局域网之外
- ▶ 要求所有进程监听传入的位置请求

地址解析协议 (ARP)

要找出哪个 MAC 地址与 IP 地址相关联，请广播查询“谁拥有这个 IP 地址”？

Forwarding pointers转发指针

当实体移动时，它会留下指向其下一个位置的指针

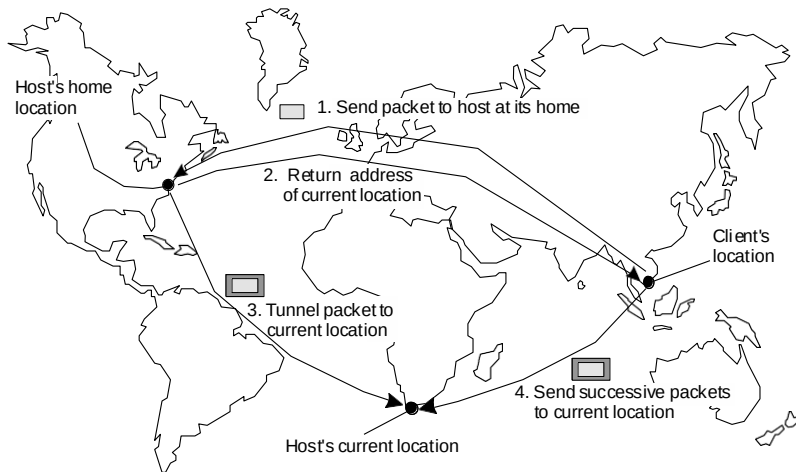
- ▶ 只需遵循指针链，就可以使解除引用对客户端完全透明
- ▶ 找到当前位置后更新客户端的引用
- ▶ 地理可扩展性问题（需要单独的链减少机制）：
 - ▶ 长链不具有容错能力
 - ▶ 解除引用时网络延迟增加

Home-based方法

单层方案：让主地址跟踪实体的位置

- ▶ 实体的主地址在命名服务上注册
- ▶ 主地址注册实体的国外地址
- ▶ 客户端首先联系主地址，然后继续联系国外位置

移动IP的原理



Home-based方法

基于home-based方法存在的问题

- ▶ 必须在实体的整个生命周期内支持 Home address
- ▶ Home address 是固定的⇒ 当实体永久移动时会产生不必要的负担
- ▶ 地理可扩展性差（实体可能位于客户端旁边）

注意

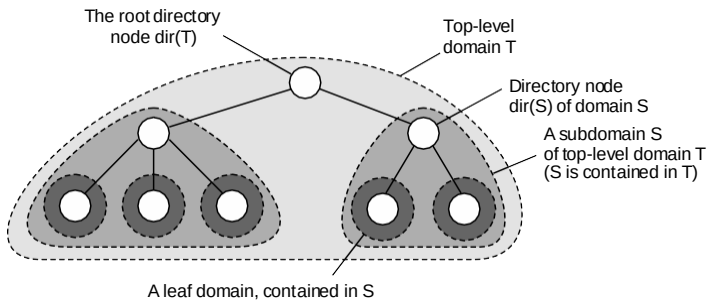
永久移动可以通过另一层命名（DNS）来解决

Hierarchical Location Services (HLS) 分层定位服务

基本思想

构建一个大规模搜索树，将底层网络划分为层次化域。每个域由一个单独的目录节点表示。

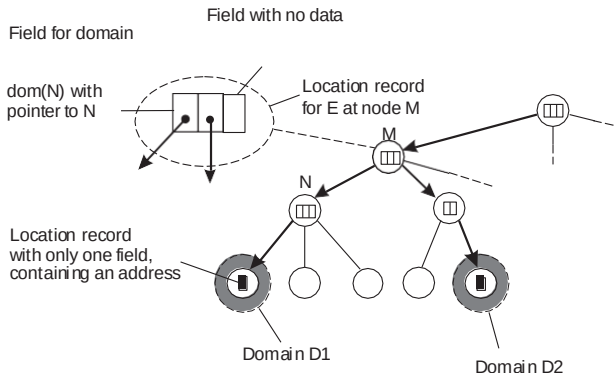
原理



HLS分层定位服务: 树组织结构

- ▶ 实体 E 的地址存储在叶节点或中间节点中
- ▶ 当且仅当以子节点为根的子树存储实体的地址时，中间节点才包含指向子节点的指针
- ▶ 根节点知道所有实体

将具有两个地址的实体的信息存储在不同的叶域中

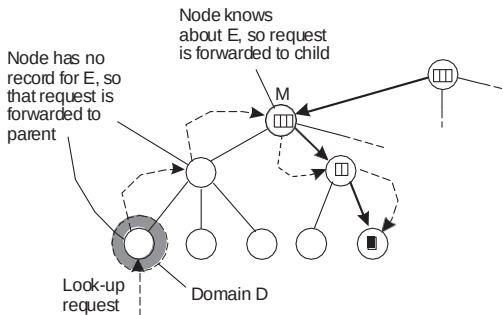


HLS分层定位服务: 查询操作

基本原则

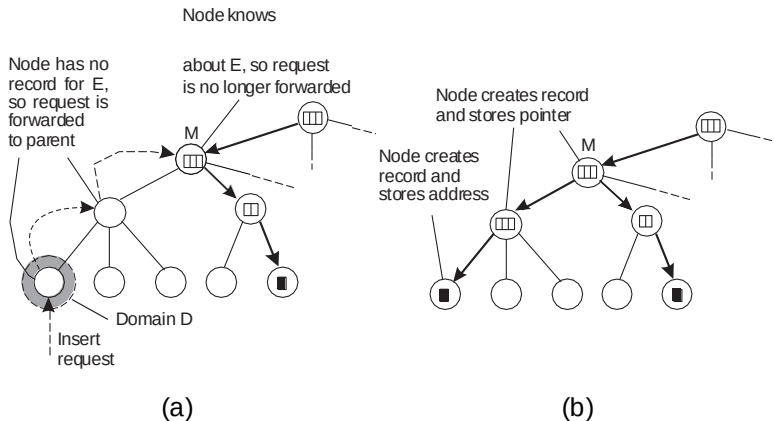
- ▶ 从本地叶节点开始查找
- ▶ 节点知道 $E \Rightarrow$ 跟随向下指针，否则向上查找
- ▶ 向上查找始终在根节点处停止

查询位置



HLS分层定位服务: 插入操作

- (a) 插入请求被转发到第一个知道实体 E 的节点。
- (b) 创建指向叶节点的转发指针链

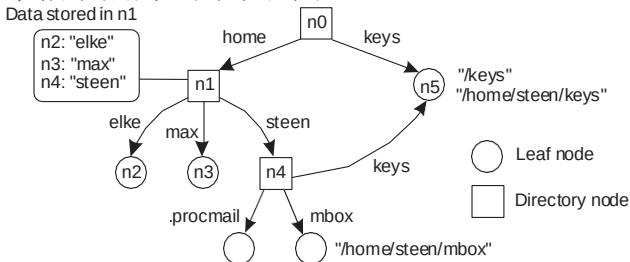


名字空间

Naming graph命名图

叶节点代表（命名）实体的图。目录节点是引用其他节点的实体。

具有单个根节点的通用命名图



注意

目录节点包含一个（节点标识符，边标签）对的表。

名字空间

我们可以轻松地在节点中存储各种属性

- ▶ 实体的类型
- ▶ 该实体的标识符
- ▶ 实体位置的地址
- ▶ 昵称
- ▶ ...

名字空间

我们可以轻松地在节点中存储各种属性

- ▶ 实体的类型
- ▶ 该实体的标识符
- ▶ 实体位置的地址
- ▶ 昵称
- ▶ ...

注意

目录节点除了存储带有（标识符，标签）对的目录表之外，还可以具有属性。

Name resolution名称解析

问题

要解析名称，我们需要一个目录节点。我们实际上如何找到该（初始）节点？

Name resolution 名称解析

问题

要解析名称，我们需要一个目录节点。我们实际上如何找到该（初始）节点？

Closure mechanism 闭包机制：选择开始名称解析的隐式上下文的机制

- ▶ `www.distributed-systems.net` : 从 DNS 名称服务器开始
- ▶ `/home/maarten/mbox` : 从本地 NFS 文件服务器开始（可能进行递归搜索）
- ▶ `0031 20 598 7784`: 拨打电话号码
- ▶ `77.167.55.6`: 将消息路由到特定 IP 地址

Name resolution名称解析

问题

要解析名称，我们需要一个目录节点。我们实际上如何找到该（初始）节点？

Closure mechanism闭包机制：选择开始名称解析的隐式上下文的机制

- ▶ `www.distributed-systems.net` : 从 DNS 名称服务器开始
- ▶ `/home/maarten/mbox` : 从本地 NFS 文件服务器开始（可能进行递归搜索）
- ▶ `0031 20 598 7784`: 拨打电话号码
- ▶ `77.167.55.6`: 将消息路由到特定 IP 地址

注意

您无法拥有明确的关闭机制——您将如何开始？

名字链接

硬链接

到目前为止，我们所描述的**路径名称**：通过遵循命名图中从一个节点到另一个节点的特定路径来解析的名称。

软链接：允许节点 **N** 包含另一个节点的**名称**

- ▶ 首先解析 **N** 的名称（导致 **N**）
- ▶ 读取 **N** 的内容，得出名称
- ▶ 名称解析继续使用名称

名字链接

硬链接

到目前为止，我们所描述的**路径名称**：通过遵循命名图中从一个节点到另一个节点的特定路径来解析的名称。

软链接：允许节点 **N** 包含另一个节点的**名称**

- ▶ 首先解析 **N** 的名称（导致 **N**）
- ▶ 读取 **N** 的内容，得出名称
- ▶ 名称解析继续使用名称

观察

- ▶ 名称解析过程决定我们读取一个节点的内容，特别是我们需要转到的另一个节点中的名称。
- ▶ 不管怎样，我们知道在哪里以及如何根据给定的名称开始名称解析

Mounting挂载

问题

名称解析还可用于通过**挂载**以透明方式合并**不同的名称空间**：将另一个名称空间的节点标识符与当前名称空间中的节点相关联。

术语

- ▶ **Foreign name space外部名称空间**：需要访问的名称空间
- ▶ **Mount point挂载点**：当前名称空间中包含外部名称空间节点标识符的节点
- ▶ **Mounting point 挂载点**：外部名称空间中继续名称解析的节点

跨网络挂载

1. 访问协议的名称。
2. 服务器的名称。
3. 外部名称空间中挂载点的名称。

名字空间实现

基本问题

通过分布命名图的节点，将名称解析过程以及名称空间管理分布到多台机器上。

名字空间实现

基本问题

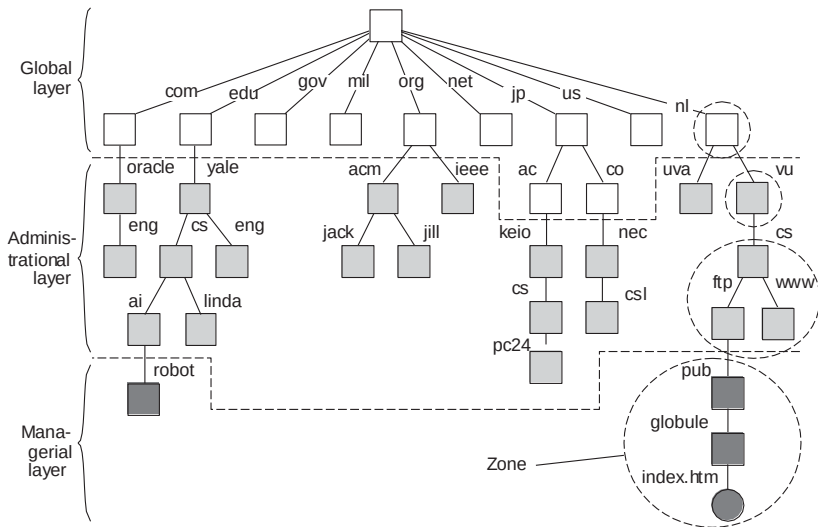
通过分布命名图的节点，将名称解析过程以及名称空间管理分布到多台机器上。

区分三个级别

- ▶ **global layer** 全局级别：由高级目录节点组成。主要方面是这些目录节点必须由不同的管理部门共同管理
- ▶ **administrational layer** 行政级别：包含中级目录节点，这些节点可以分组，以便每个组可以分配给单独的管理部门。
- ▶ 管理级别：由单个管理部门内的低级目录节点组成。主要问题是有效地将目录节点映射到本地名称服务器。

名字空间实现

名称空间分区示例，包括网络文件



名字空间实现

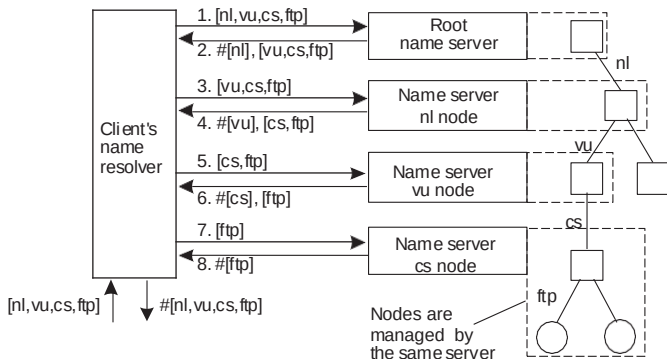
用于在名称空间中实现节点的名称服务器之间的比较

Item	Global	Administrational	Managerial
1	Worldwide	Organization	Department
2	Few	Many	Vast numbers
3	Seconds	Milliseconds	Immediate
4	Lazy	Immediate	Immediate
5	Many	None or few	None
6	Yes	Yes	Sometimes
1: Geographical scale 2: # Nodes 3: Responsiveness		4: Update propagation 5: # Replicas 6: Client-side caching?	

迭代名字解析

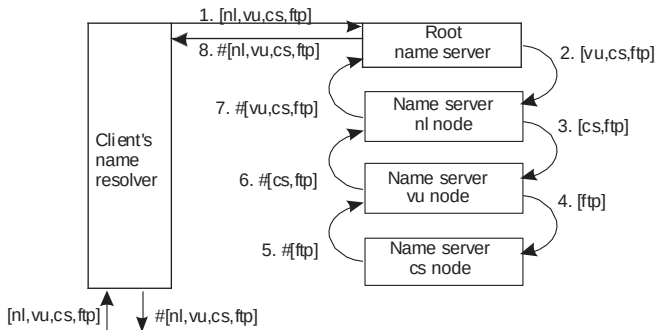
原则

1. $resolve(dir, [name_1, \dots, name_K])$ sent to $Server_0$ responsible for dir
2. $Server_0$ resolves $resolve(dir, name_1) \rightarrow dir_1$, returning the identification (address) of $Server_1$, which stores dir_1 .
3. Client sends $resolve(dir_1, [name_2, \dots, name_K])$ to $Server_1$, etc.



递归名称解析 原则

1. $resolve(dir, [name_1, \dots, name_K])$ sent to $Server_0$ responsible for dir
2. $Server_0$ resolves $resolve(dir, name_1) \rightarrow dir_1$, and sends $resolve(dir_1, [name_2, \dots, name_K])$ to $Server_1$, which stores dir_1 .
3. $Server_0$ waits for result from $Server_1$, and returns it to client.



Caching in recursive name resolution

Recursive name resolution of *[nl, vu, cs, ftp]*

Server for node	Should resolve	Looks up	Passes to child	Receives and caches	Returns to requester
<i>cs</i>	<i>[ftp]</i>	<i>#[ftp]</i>	—	—	<i>#[ftp]</i>
<i>vu</i>	<i>[cs, ftp]</i>	<i>#[cs]</i>	<i>[ftp]</i>	<i>#[ftp]</i>	<i>#[cs]</i> <i>#[cs, ftp]</i>
<i>nl</i>	<i>[vu, cs, ftp]</i>	<i>#[vu]</i>	<i>[cs, ftp]</i>	<i>#[cs]</i> <i>#[cs, ftp]</i>	<i>#[vu]</i> <i>#[vu, cs]</i> <i>#[vu, cs, ftp]</i>
<i>root</i>	<i>[nl, vu, cs, ftp]</i>	<i>#[nl]</i>	<i>[vu, cs, ftp]</i>	<i>#[vu]</i> <i>#[vu, cs]</i> <i>#[vu, cs, ftp]</i>	<i>#[nl]</i> <i>#[nl, vu]</i> <i>#[nl, vu, cs]</i> <i>#[nl, vu, cs, ftp]</i>

基于属性的命名

Observation

在许多情况下，通过实体的属性⇒传统目录服务（又称黄页）来命名和查找实体要方便得多。

基于属性的命名

Observation

在许多情况下，通过实体的属性⇒传统目录服务（又称黄页）来命名和查找实体要方便得多。

问题

查找操作可能非常昂贵，因为它们需要将请求的属性值与实际属性值进行匹配⇒检查所有实体（原则上）。

实现目录服务

可扩展搜索解决方案

将基本目录服务实现为数据库，并与传统结构化命名系统相结合。

轻量级目录访问协议 (LDAP)

每个目录条目由（属性、值）对组成，并具有唯一名称以方便查找。

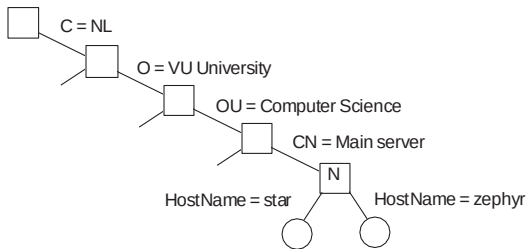
Attribute	Abbr.	Value
Country	C	NL
Locality	L	Amsterdam
Organization	O	VU University
OrganizationalUnit	OU	Computer Science
CommonName	CN	Main server
Mail_Servers	–	137.37.20.3, 130.37.24.6, 137.37.20.10
FTP_Server	–	130.37.20.20
WWW_Server	–	130.37.20.20

LDAP

本质

- ▶ 目录信息库：LDAP 服务中所有目录条目的集合。
- ▶ 每条记录都以命名属性序列（称为相对可分辨名称）唯一命名，以便查找。
- ▶ 目录信息树：LDAP 目录服务的命名图；每个节点代表一个目录条目。

目录信息树的一部分



分布式索引

基本思想

- ▶ Assume a set of attributes $\{a^1, \dots, a^N\}$
- ▶ Each attribute a^k takes values from a set R^k
- ▶ For each attribute a^k associate a set $\mathbf{S}^k = \{S_1^k, \dots, S_{n_k}^k\}$ of n_k servers
- ▶ **Global mapping** $F: F(a^k, v) = S_j^k$ with $S_j^k \in \mathbf{S}^k$ and $v \in R^k$

观察

If $L(a^k, v)$ is set of keys returned by $F(a^k, v)$, then a query can be formulated as a logical expression, e.g.,

$$F(a^1, v^1) \wedge F(a^2, v^2) \vee F(a^3, v^3)$$

which can be processed by the client by constructing the set

$$L(a^1, v^1) \cap L(a^2, v^2) \cup L(a^3, v^3)$$

分布式索引缺点

Quite a few

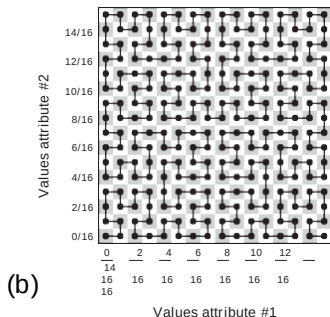
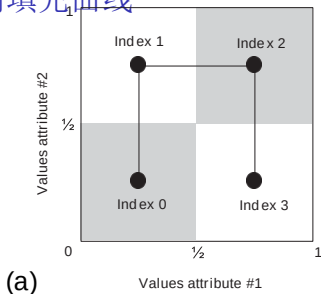
- ▶ A query involving k attributes requires contacting k servers
- ▶ Imagine looking up "*lastName = Smith $\wedge$ firstName = Pheriby*": the client may need to process **many** files as there are so many people named "Smith."
- ▶ No (easy) support for **range queries**, such as "*price = [1000 - 2500]*."

替代方案：将所有属性映射到 1 维，然后进行索引

空间填充曲线：原理

1. Map the N -dimensional space covered by the N attributes $\{a^1, \dots, a^N\}$ into a single dimension
2. Hashing values in order to distribute the 1-dimensional space among index servers.

a) 1 阶希尔伯特空间填充曲线， (b) 4 阶希尔伯特空间填充曲线



空间填充曲线

Once the curve has been drawn

Consider the two-dimensional case

- ▶ a Hilbert curve of order k connects 2^{2k} subsquares $\Rightarrow$ has 2^{2k} indices.
- ▶ A range query corresponds to a **rectangle** R in the 2-dimensional case
- ▶ R intersects with a number of subsquares, each one corresponding to an index $\Rightarrow$ we now have a **series of indices** associated with R .

Getting to the entities

Each index is to be mapped to a server, who keeps a reference to the associated entity. One possible solution: **use a DHT**.